

jordanvalnet/exalthon26

Onboard : un pipeline d'agents bien contraint, à adopter en pilote interne, pas à forker tant qu'il n'a ni licence ni second mainteneur

Hackathon eXaltemps 2026 : agent + MCP GitHub

1

BUS FACTOR
sur 12 semaines

58

COMMITTS
sur 12 semaines

3

DÉPENDANCES
1 en production

aucune

LICENCE

<https://github.com/jordanvalnet/exalthon26> Sans licence Onboard · profil cto

Que fait ce projet et quelle est son architecture ?

Un pipeline d'agents à validation dure entre chaque étape, dont le cœur est du texte versionné (prompts, cache) et non un service : auditable en une heure, repreneable sans l'auteur ^{1 2}.

<code>.claude/</code>	skills et réglages Claude Code
<code>chat/</code>	client du chat d'équipe
<code>docs/</code>	documentation du projet
<code>onboard/</code>	pipeline Onboard : agents, profils, cache
<code>onboarding/</code>	second package bun : collecteurs MCP
<code>pitch/</code>	supports du pitch
<code>src/</code>	code principal : CLI, collecte, rendu, validation
<code>test/</code>	2 tests bun
<code>.env.example</code>	modèle de variables d'environnement
<code>.gitignore</code>	exclusions git
<code>.mcp.json</code>	config du serveur MCP GitHub
<code>AGENTS.md</code>	consignes pour les agents IA
<code>CIBLES.md</code>	repos cibles du hackathon
<code>CLAUDE.md</code>	instructions Claude Code, 69 octets
<code>HACKATHON.md</code>	consignes du jury
<code>LIVRABLE.md</code>	rapport de mission livré

... et 5 autres entrées, voir `sources/tree.md`

Onboard parcourt n'importe quel dépôt GitHub par le serveur MCP GitHub, construit un cache documentaire vérifié et en tire une présentation par public : dev, qa, cto, ceo, investisseur, enfant. Chaque chiffre renvoie à sa source, et un guide répond ensuite aux questions à partir du cache ^{1 3}.

Livrable d'un hackathon d'une journée : dépôt créé le 2026-09-09, dernier push le 2026-09-10, propriétaire individuel, branche `main` , 0 étoile, 0 fork ^{4 5 6 7 8 9}.

Architecture en quatre étapes chaînées par fichiers : collecter (5 sous-agents en parallèle, outils `github:*`), rédiger (lit le cache, jamais GitHub), rendre (code, une identité visuelle par profil), guider (réponse citée, FAQ enrichie). Chaque étape ne lit que les sorties de la précédente et ne démarre que si une validation en code passe ¹.

21 entrées à la racine, 8 dossiers : `src/` (CLI, collecte, rendu, validation), `onboard/` (prompts des agents, profils JSON, caches versionnés), `onboarding/` (second package bun, collecteurs MCP), `chat/` (client du chat d'équipe), `test/` (2 tests bun) ^{10 11 12 13 14 15}.

Choix structurant : les prompts vivent dans `onboard/agents/` , en texte, pour que Claude Code, Copilot et Codex exécutent le même texte ; les validations sont du code, `src/validate.ts` , `src/merge.ts` , schéma zod `src/facts.ts` . Un seul chemin vers GitHub, le serveur MCP, jamais REST ^{1 16}.

Quelle est sa santé technique ?

Code discipliné mais sans filet automatique : zéro CI, zéro release, 2 tests ; la qualité repose sur `bun run check` lancé à la main ^{17 18 19}.



58 commits sur 12 semaines, dont 57 la semaine du hackathon (2026-W37) et 1 en 2026-W35 : toute l'activité tient en deux jours, dernier commit le 2026-09-10 ^{20 21 22}.

Aucune release, aucun tag : le seul numéro de version est le `0.1.0` du manifeste, et le package est `private` ^{19 23}.

Aucune intégration continue : pas de dossier `.github`, donc aucun workflow GitHub Actions. La seule vérification est locale, `bun run check`, soit `tsc --noEmit` puis `bun test` ^{17 24}.

Tests : 2 fichiers dans `test/`, `github.test.ts` et `issues.test.ts`, framework `bun test` ; à en juger par leurs noms, ils couvrent l'accès GitHub et la collecte des issues, pas le rendu ni les validations ^{18 25 26}.

Ce qui est solide : TypeScript strict, bun sans npm, un seul chemin vers GitHub, une validation en code à chaque étape, et 4 repères de documentation IA sur 7 (AGENTS.md, CLAUDE.md, .mcp.json, .claude/skills/) ^{16 2 27}.

```
bun install
bun onboard owner/repo profil
bun run check
```

Les trois commandes du README : installer, lancer, vérifier ^{28 29 30}.

Qui porte le projet et quel est le bus factor ?

Bus factor 1, 41 commits sur 58 pour une personne : sans second mainteneur nommé, le projet s'arrête avec son auteur ^{31 32}.



5 contributeurs humains sur 12 semaines : jordanvalnet 41 commits, PrincyExaltIT 8, Sacane 4, radomd92 3, loic.vyncke 2 ^{33 32 34 35 36 37}.

Bus factor 1 : une seule personne signe plus de la moitié des 58 commits, plus que les quatre autres réunies ^{31 38}.

Aucun CODEOWNERS, aucune politique de sécurité, aucun financement déclaré : la gouvernance est implicite, portée par l'équipe du hackathon ^{39 40 41}.

À lire avec le contexte : une équipe constituée pour une journée (« Équipe eXaltemps, hackathon du 9 septembre 2026 »), pas une communauté ; la concentration des commits est celle d'un intégrateur qui assemble le travail des autres, ce qui ne dit rien de la capacité de chacun à reprendre le code ^{1 4}.

Parade : un second mainteneur avec droits d'écriture et un fichier CODEOWNERS sur `src/` et `onboard/` avant tout usage au-delà du pilote ^{31 39}.

Quelles dépendances et quelle dette ?

Dette de dépendances quasi nulle (1 runtime), mais dette d'exploitation réelle : le pipeline dépend d'un agent IA tiers et d'un serveur MCP hébergé, hors de votre contrôle ^{42 1}.

3 dépendances déclarées dans `package.json` , `bun.lock` versionné à la racine ^{42 43 23}.

DÉPENDANCE	TYPE	VERSION	REMARQUE
zod	runtime	^4.5.4	schéma de facts.json, seule dépendance d'exécution ^{44 45}
typescript	dev	^5	typecheck <code>tsc --noEmit</code> ²³
@types/bun	dev	latest	non épinglée : le typecheck peut casser sans changement du code ²³

Dépendances hors manifeste, invisibles au lock : le runtime bun, l'assistant Claude Code que `bun onboard` lance en non interactif, le serveur MCP GitHub hébergé `https://api.githubcopilot.com/mcp/` , et Chrome ou Edge en headless pour les PDF ^{1 16 46}.

Un second `onboarding/package.json` (379 octets) coexiste avec le principal : deux packages bun dans un même dépôt, deux arbres de dépendances à suivre ^{10 47}.

Dette repérée : 12 résultats pour « TODO » dans le dépôt, 10 listés, un seul dans un fichier de code (`onboarding/src/report/skeleton.ts`), les 9 autres dans la doc et les caches, où c'est un mot du texte ⁴⁸.

Quels risques ?

Deux risques bloquants pour un fork ou un usage externe, licence absente et bus factor 1 ; les trois autres sont des fichiers à ajouter : un workflow, un SECURITY.md, une version épinglée ^{49 50 51}.

- license** Pas de fichier LICENSE : réutilisation juridiquement incertaine.
- security** Pas de SECURITY.md : aucun canal déclaré pour signaler une faille.
- bus_factor** Un seul contributeur porte la moitié des commits sur 12 semaines.
- ci** Pas de dossier .github à la racine (21 entrées listées) : aucun workflow GitHub Actions. La seule vérification est locale, ``bun run check` = `tsc --noEmit && bun test`` (package.json).
- deps** 3 dépendances déclarées dans package.json : 1 runtime (zod ^4.5.4), 2 dev (@types/bun latest, typescript ^5) ; bun.lock versionné (1355 octets). Second package.json dans onboarding/ (379 octets), non lu.

Cinq risques collectés, deux au niveau high, chacun avec sa parade ⁵².

RISQUE	NIVEAU	CONSTAT	PARADE
Licence	HIGH	Pas de fichier LICENSE : réutilisation juridiquement incertaine ^{49 53}	Obtenir de l'auteur un LICENSE (MIT ou Apache-2.0) avant tout fork ou usage hors de l'équipe
Bus factor	HIGH	Une personne porte la moitié des commits sur 12 semaines ^{50 38}	Second mainteneur, CODEOWNERS
CI	MID	Aucun workflow GitHub Actions, vérification locale seule ^{51 54}	Un workflow qui lance <code>bun run check</code> à chaque push
Sécurité	MID	Pas de SECURITY.md, aucun canal de signalement ^{55 56}	Un SECURITY.md avec un contact
Dépendances	LOW	3 déclarées, lock versionné ^{57 47}	Épingler <code>@types/bun</code>

Le cache ne contient pas `security.md` : le dépôt n'a pas de politique de sécurité ; à vérifier dans <https://github.com/jordanvalnet/exalthon26/security/policy> ^{40 58}.

Non mesurable par la collecte : les CVE des dépendances et la répartition des langages, que le serveur MCP n'expose pas ; à vérifier dans <https://github.com/jordanvalnet/exalthon26/security/dependabot> ⁵⁹.

Risque propre à l'architecture : le secret `GITHUB_PAT` (scope repo) part vers un serveur MCP hébergé par GitHub, en en-tête `Authorization: Bearer` ; à cadrer par un jeton à granularité fine, limité aux dépôts à onboarder ^{16 1}.

Adopter, contribuer ou forker ?

Adopter en pilote, contribuer si l'équipe reste active, ne pas forker : le blocage est juridique, pas technique ^{49 1}.

Feuille de route : aucune PR ouverte, aucun thème étiqueté ; la seule issue ouverte, #1 «  Chat équipe — canal IA ↔ IA » (22 commentaires), est le canal où les assistants IA de l'équipe se parlent, pas une demande produit ^{60 61 62 63 16}.

Pas de ROADMAP.md ni de docs/roadmap.md dans le dépôt, et les jalons ne sont pas exposés par le serveur MCP ; à vérifier dans <https://github.com/jordanvalnet/exalthon26/milestones> ^{64 65}.

58 commits pour 1 PR fusionnée sur 30 jours, #2 « Onboard : workflow agentique, contrat du cache, profils, agents, validations », le 2026-09-09 : le travail passe par des commits directs sur `main`, pas par relecture ^{66 67 68 20}.

Adopter : oui, en pilote interne sur vos propres dépôts. Le produit est utilisable en trois commandes, le cache est du texte que l'on peut relire et corriger, et six profils sont livrés ^{1 29}.

Contribuer : le dépôt s'y prête, prompts en texte dans `onboard/agents/`, profils JSON, consignes pour les agents dans AGENTS.md ; mais pas de CONTRIBUTING.md à la racine ni de good first issue ^{10 69 70}.

Forker : non, tant qu'il n'y a pas de LICENSE : sans licence, aucun droit de copie ni de redistribution n'est accordé ^{53 49}.

Décision

Recommandation : **adopter en pilote interne**, sur vos propres dépôts, sans fork ni redistribution, jusqu'à la levée de deux conditions, la licence et le bus factor ^{49 50}.

1. Licence : obtenir de l'auteur un fichier LICENSE ; sans lui, aucun droit de réutilisation n'est acquis, même en interne ^{53 10}.
2. Bus factor : nommer un second mainteneur et un CODEOWNERS ; aujourd'hui une personne signe 41 commits sur 58 ^{31 32 39}.
3. Filet minimal avant tout usage régulier : un workflow qui lance `bun run check`, un SECURITY.md, `@types/bun` épinglé, et un `GITHUB_PAT` à granularité fine limité aux dépôts onboardés ^{17 40 23 16}.

Ce que vous achetez : une architecture lisible, des prompts et un cache en texte versionné, une seule dépendance runtime ; ce que vous n'achetez pas encore : une garantie de maintenance ^{1 42 31}.

NOTES

Sources

1. source du repo `readme.md`
2. donnée collectée `ai_docs.score`
3. donnée collectée `repo.description`
4. donnée collectée `repo.created_at`
5. donnée collectée `repo.pushed_at`
6. donnée collectée `repo.owner_type`
7. donnée collectée `repo.default_branch`
8. donnée collectée `repo.stars`
9. donnée collectée `repo.forks`
10. source du repo `tree.md`
11. donnée collectée `tree.18.role`
12. donnée collectée `tree.13.role`
13. donnée collectée `tree.14.role`
14. donnée collectée `tree.11.role`
15. donnée collectée `tree.19.role`
16. source du repo `ai-docs.md`
17. donnée collectée `build.ci`
18. donnée collectée `tests.files`
19. donnée collectée `releases`
20. donnée collectée `activity.commits_per_week.11.count`
21. donnée collectée `activity.commits_per_week.9.count`
22. donnée collectée `activity.last_commit`
23. source du repo `manifest.md`
24. source du repo `ci.md`
25. donnée collectée `tests.framework`
26. source du repo `tests.md`
27. donnée collectée `ai_docs.max`
28. donnée collectée `build.install`
29. donnée collectée `build.run`
30. donnée collectée `build.test`
31. donnée collectée `activity.bus_factor`
32. donnée collectée `activity.contributors.0.commits`
33. donnée collectée `activity.contributors.0.login`
34. donnée collectée `activity.contributors.1.commits`
35. donnée collectée `activity.contributors.2.commits`
36. donnée collectée `activity.contributors.3.commits`
37. donnée collectée `activity.contributors.4.commits`
38. donnée collectée `risks.2.note`
39. donnée collectée `business.codeowners`
40. donnée collectée `business.security_policy`
41. donnée collectée `business.funding`
42. donnée collectée `deps.count`
43. donnée collectée `deps.manifest`
44. donnée collectée `deps.runtime.0.name`
45. donnée collectée `deps.runtime.0.version`
46. donnée collectée `entrypoints.1.why`
47. donnée collectée `risks.4.note`
48. source du repo `todo.md`
49. donnée collectée `risks.0.level`
50. donnée collectée `risks.2.level`
51. donnée collectée `risks.3.level`
52. donnée collectée `risks`
53. donnée collectée `risks.0.note`
54. donnée collectée `risks.3.note`
55. donnée collectée `risks.1.level`
56. donnée collectée `risks.1.note`
57. donnée collectée `risks.4.level`
58. GitHub <https://github.com/jordanvalnet/exalthon26/security/policy>
59. GitHub <https://github.com/jordanvalnet/exalthon26/security/dependabot>
60. donnée collectée `roadmap.open_prs`
61. donnée collectée `roadmap.themes`
62. donnée collectée `roadmap.requests.0.title`
63. donnée collectée `roadmap.requests.0.comments`
64. source du repo `docs.md`
65. GitHub <https://github.com/jordanvalnet/exalthon26/milestones>
66. donnée collectée `pulls.merged_30d.0.title`
67. donnée collectée `pulls.merged_30d.0.merged_at`
68. donnée collectée `pulls.open`
69. donnée collectée `ai_docs.files.0.path`
70. donnée collectée `issues.good_first`